

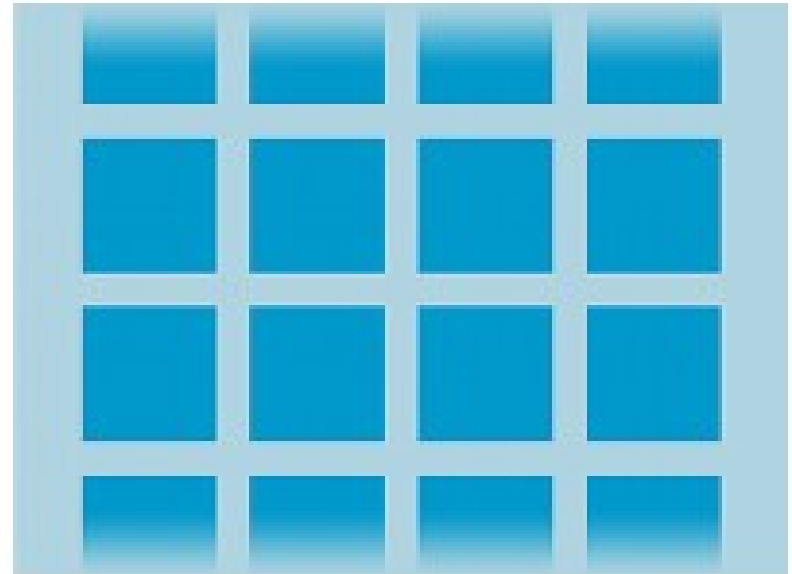
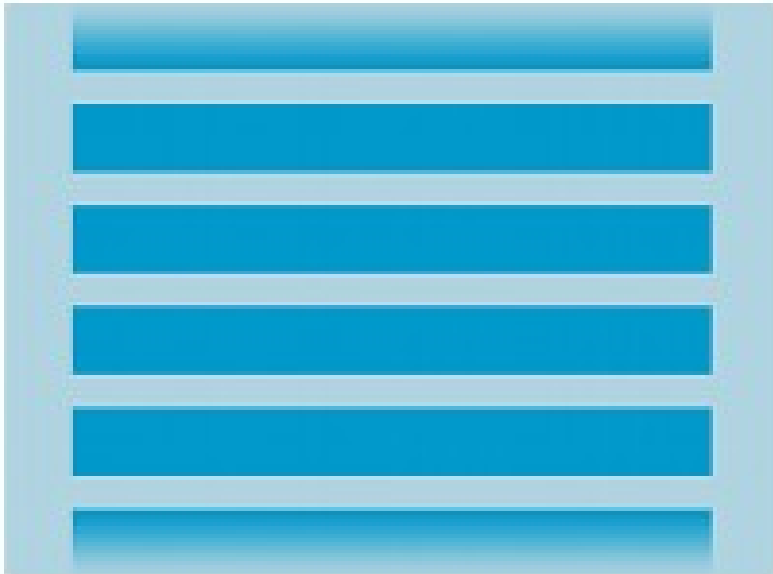
Programação

Dispositivos Móveis

aula 09

- Quando o conteúdo do layout é dinâmico ou não pré-determinado, é possível usar a classe "AdapterView" para preencher o layout com exibições em tempo de execução.
- O "Adapter" se comporta como um intermediário entre a fonte dos dados e o layout do "AdapterView".
- O "Adapter" recupera os dados (de uma fonte como uma matriz ou uma consulta de banco de dados) e converte cada entrada em uma exibição que pode ser adicionada ao layout do "AdapterView";

- Alguns layouts comuns retornados por um "Adapter":
 - ListView: Exibe uma lista de rolagem de coluna única
 - GridView: Exibe uma grade de rolagem de colunas e linhas.



- É possível preencher um "AdapterView" como um layout do tipo "ListView" ou "GridView" vinculando a instância de "AdapterView" a um "Adapter", que recupera dados de uma fonte externa e cria uma View que representa cada entrada de dados.
- O Android oferece diversas subclasses de "Adapter" que são úteis para recuperar diferentes tipos de dados e criar exibições de um "AdapterView".
- Alguns tipos de "Adapters":
 - BaseAdapter
 - ArrayAdapter
 - SimpleCursorAdapter

- A classe “Adapter” é criada para fornecer conteúdo ao ListView;
- Para criarmos um “Adapter” customizado, devemos criar uma classe que herde de “BaseAdapter”, implementar alguns métodos e usar uma instância dessa classe em algum componente;

- BaseAdapter é uma classe abstrata, que já traz os métodos abstratos necessários à exibição de uma lista na tela;
- Métodos:
 - `public int getCount()` : Indica quantos itens temos para exibir na listagem. Com isso, o Android calcula o tamanho inicial da ListView;
 - `public long getItemId(int posicao)` : Retorna um identificador único para o item da lista;
 - `public Object getItem(int posicao)` : Retorna um item da lista, de acordo com a posição;
 - `public View getView(int posicao, View convertView, ViewGroup parent)` : Retorna um objeto View, representando a linha da ListView que será exibida na tela do dispositivo.

- Esse adaptador é usado quando a fonte de dados for uma matriz.
- Por padrão, ArrayAdapter cria uma exibição para cada item de matriz chamando o método "toString()" em cada item e posicionando o conteúdo em um componente "TextView".
- Por exemplo, se você tiver uma matriz de strings que deseja exibir em uma ListView, inicialize um novo ArrayAdapter usando um construtor para especificar o layout de cada string e a matriz de strings:

```
ArrayAdapter<String> adapter = new ArrayAdapter<String>(this,  
    android.R.layout.simple_list_item_1, myStringArray);
```

- Os argumentos desse construtor são:
 - Context do aplicativo
 - Layout que contém uma TextView para cada string na matriz
 - A matriz de strings
- Em seguida, simplesmente chame `setAdapter()` na `ListView`:

```
ArrayAdapter<String> adapter = new ArrayAdapter<String>(this,  
    android.R.layout.simple_list_item_1, myStringArray);
```

```
ListView listView = (ListView) findViewById(R.id.listview);  
listView.setAdapter(adapter);
```

- Para personalizar a aparência de cada item, é possível modificar o método "toString()" para os objetos na matriz.
- Ou, para criar uma exibição para cada item diferente de um componente TextView (por exemplo, se você quiser uma ImageView para cada item da matriz), estenda a classe ArrayAdapter e modifique getView() para retornar o tipo de exibição que deseja para cada item.

- O ListView é um dos componentes de interface gráfica mais utilizados no desenvolvimento de aplicativos para o Android;
- Através do ListView listamos os dados para os nossos usuários, tais como: uma lista de carros, lista de pessoas, amigos no facebook, aniversariantes do mês, etc;

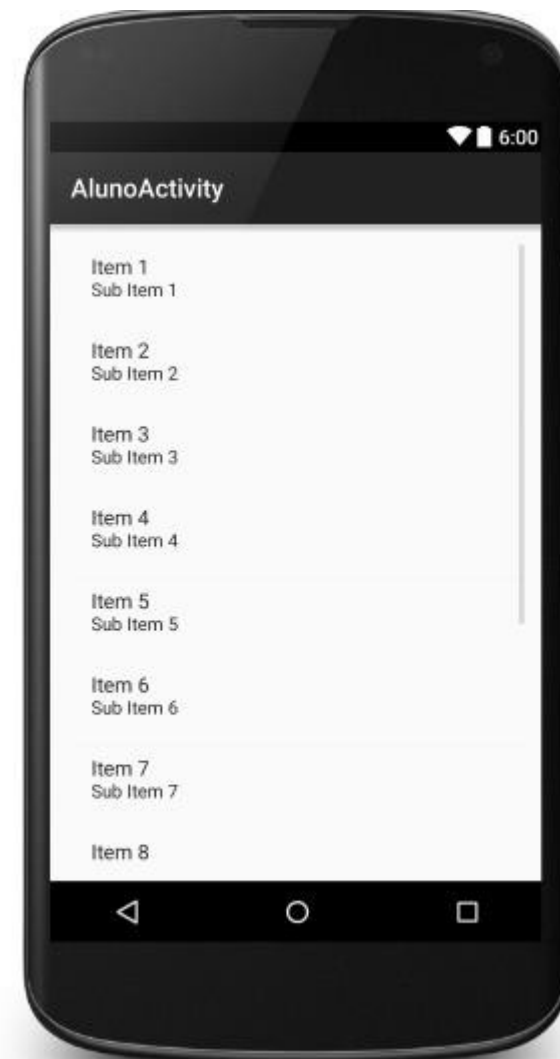
- Exibe uma lista de itens de rolagem. Os itens da lista são inseridos automaticamente à lista usando um **adapter**
- **Procedimento:** Preencher uma lista de objetos que contém os dados de seu item e passar essa lista para um **adapter** que irá preencher o ListView.
- Através desse **adapter** nós poderemos ter o controle sobre a lista, podendo remover item, adicionar e/ou pegar o item selecionado.

- É possível adicionar um listview através da inserção do seguinte xml:

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools" android:layout_width="match_parent"
    android:layout_height="match_parent" android:paddingLeft="16dp"
    android:paddingRight="16dp"
    android:paddingTop="16dp"
    android:paddingBottom="16dp"
    tools:context="com.example.matth.topaula1.AlunoActivity">

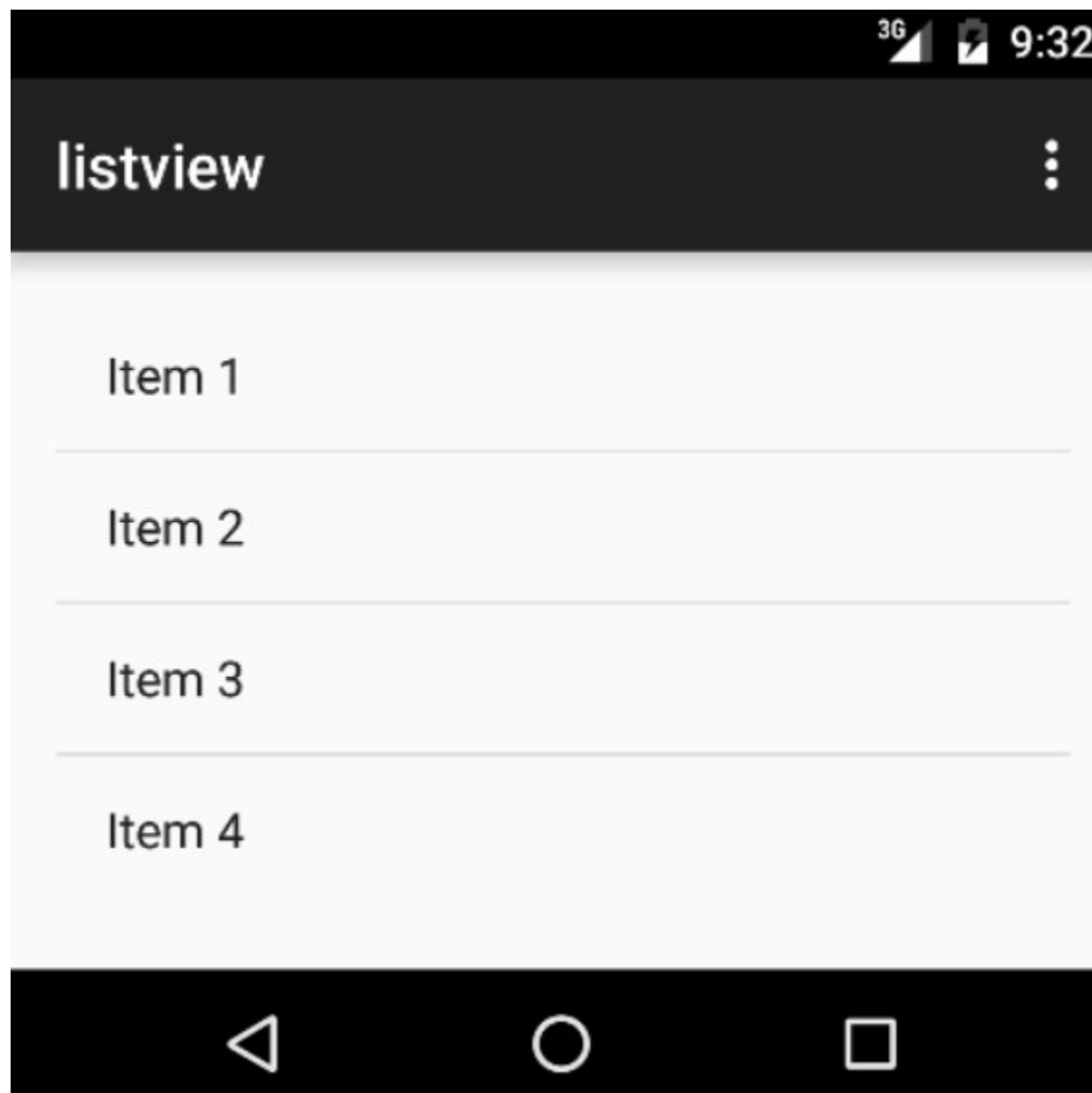
    <ListView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:id="@+id/listView"
        android:layout_alignParentTop="true"
        android:layout_alignParentLeft="true"
        android:layout_alignParentStart="true" />

</RelativeLayout>
```



- A forma mais simples de povoar um ListView é através de uma Lista de Strings (List<String>)

```
ListView listview_teste = (ListView) findViewById(R.id.listView);
List<String> list = new ArrayList<String>();
list.add("Item 1");
list.add("Item 2");
list.add("Item 3");
list.add("Item 4");
ArrayAdapter<String> dataAdapter = new ArrayAdapter<String>
    (this, android.R.layout.simple_list_item_1, list);
dataAdapter.setDropDownViewResource
    (android.R.layout.simple_dropdown_item_1line);
listview_teste.setAdapter(dataAdapter);
```



```
package com.example.matth.topaula1;

import android.content.Context;
import android.view.View;
import android.view.ViewGroup;
import android.widget.BaseAdapter;
import android.widget.TextView;

public class SimpleAdapter extends BaseAdapter {
    private Context context;
    public SimpleAdapter(Context context) {
        super();
        this.context = context;
    }
    @Override
    public int getCount() {
        return 10;
    }
}
```

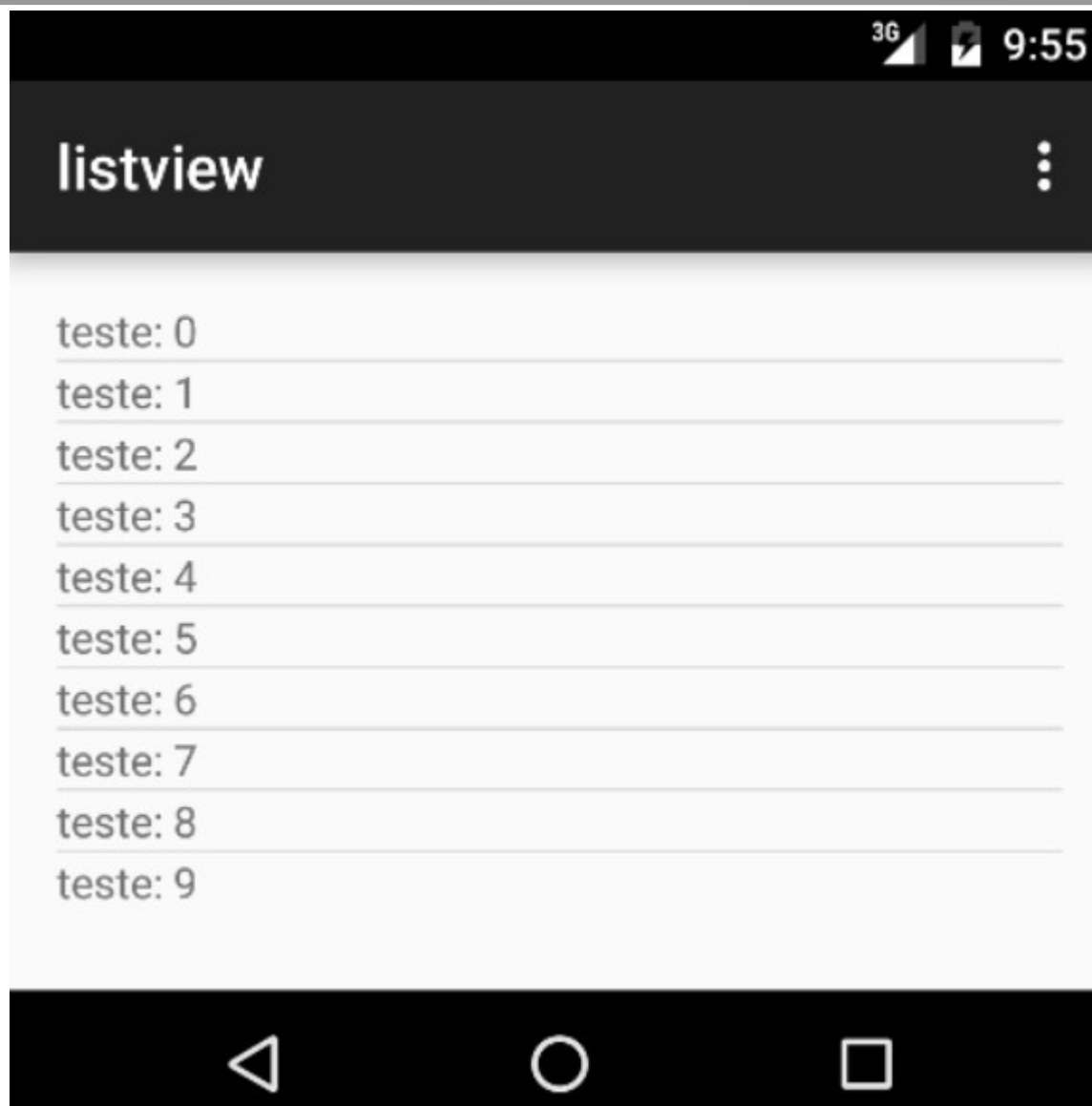
```
@Override
public Object getItem(int position) {
    return position;
}

@Override
public long getItemId(int position) {
    return position;
}

@Override
public View getView(int position, View convertView, ViewGroup parent)
{
    TextView t = new TextView(context);
    t.setText("teste: " + position );
    return t;
}
}
```

Ligando o ListView no Adapter

```
public class listview extends AppCompatActivity {  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_listview);  
        ListView listview_teste = (ListView) findViewById(R.id.listView);  
        listview_teste.setAdapter(new SimpleAdapter(this));  
    }  
}
```



```
public class listview extends AppCompatActivity implements
    AdapterView.OnItemClickListener {

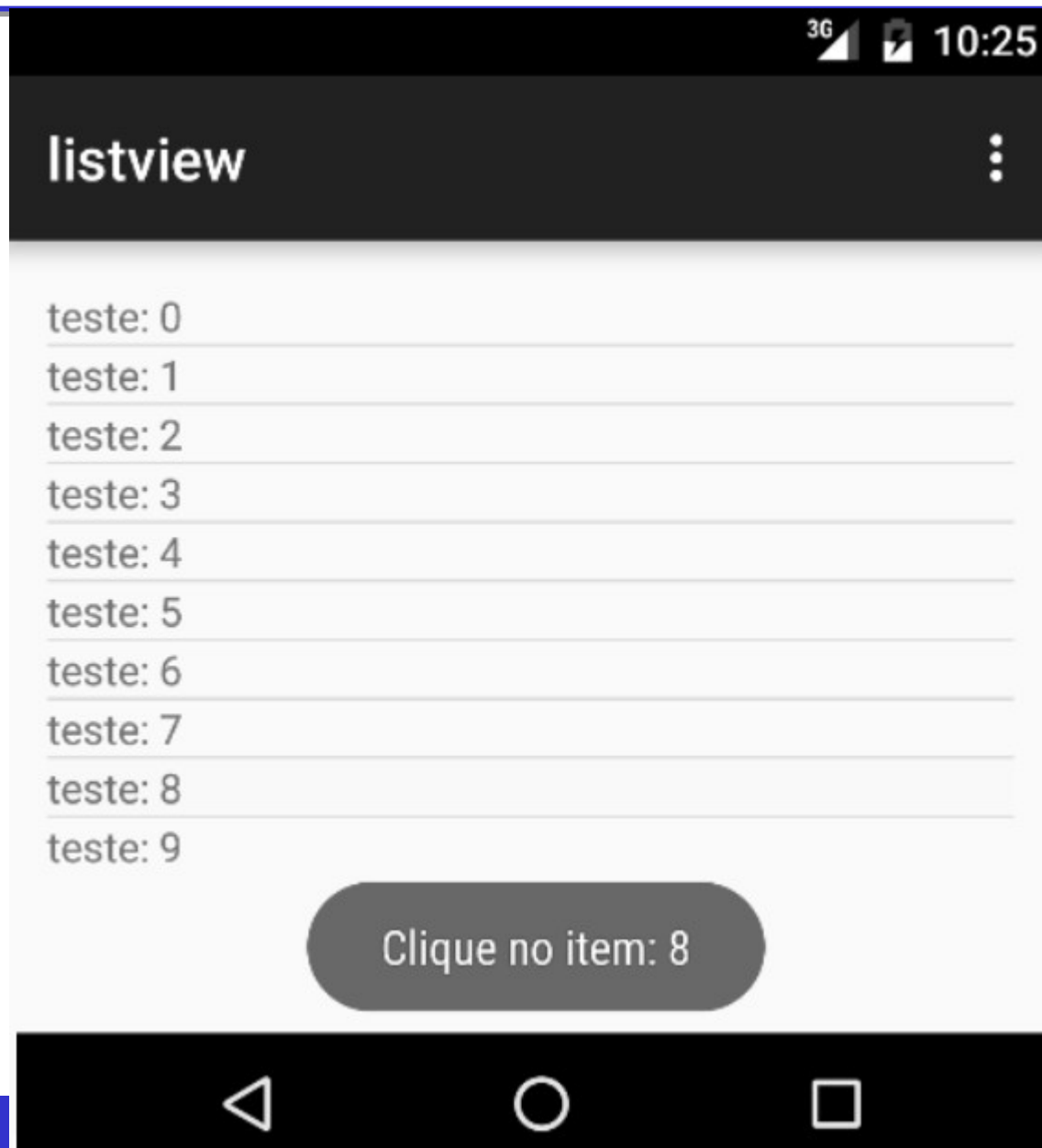
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_listview);
        ListView listview_teste = (ListView) findViewById(R.id.listView);
        listview_teste.setAdapter(new SimpleAdapter(this));
        listview_teste.setOnItemClickListener(this);
    }

    @Override
    public void onItemClick(AdapterView<?> parent, View view,
        int position, long id) {
        Toast.makeText(this, "Clique no item: " + position,
            Toast.LENGTH_SHORT).show();
    }
}
```

- O Toast fornece um feedback simples e rápido ao usuário em uma pequena janela.
- Essa janela é exibida na quantidade de espaço necessário para a mensagem que lhe foi determinada, permanecendo visível por um tempo limite.

```
Context contexto = getApplicationContext();  
String texto = "exemplo toast";  
int duracao = Toast.LENGTH_SHORT;  
Toast toast = Toast.makeText (contexto, texto, duracao);  
toast.show();
```

ListView - Tratando evento "onItemClickListener"



- Criar um Adapter para exibir uma lista objetos
- Usaremos a classe aluno:

```

1  package com.example.a364671.listadealunos;
2
3  public class Aluno {
4      private String nome;
5      public Aluno() {
6          nome="";
7      }
8      public String getNome() {
9          return nome;
10     }
11     public void setNome(String nome) {
12         this.nome = nome;
13     }
14 }
15

```

- Criando um adapter para retornar os alunos no ListView
- Atributos serão o context e a lista de alunos
- O construtor recebe a lista de alunos e o context

```
1 package com.example.a364671.listadealunos;
2
3 import android.graphics.Color;
4 import android.view.LayoutInflater;
5 import android.widget.BaseAdapter;
6 import android.content.Context;
7 import android.view.View;
8 import android.view.ViewGroup;
9 import android.widget.BaseAdapter;
10 import android.widget.ImageView;
11 import android.widget.LinearLayout;
12 import android.widget.TextView;
13
14 import java.util.List;
15
```

```

15
16 public class AlunoAdapter extends BaseAdapter {
17     private Context context;
18     private final List<Aluno> alunos;
19
20     public AlunoAdapter(Context context, List<Aluno> alunos) {
21         super();
22         this.context = context;
23         this.alunos = alunos;
24     }
25
26     @Override
27     public int getCount() { return alunos.size(); }
28
29
30
31     @Override
32     public Object getItem(int position) { return alunos.get(position); }
33
34
35
36     @Override
37     public long getItemId(int position) { return position; }
38
39
40
41     @Override
42     public View getView(int position, View convertView, ViewGroup parent) {
43         TextView t = new TextView(context);
44         Aluno a = alunos.get(position);
45         t.setText(a.getNome());
46         return t;
47     }
48 }

```

Criando o Listview com Adapter de Alunos

```
public class AlunoActivity extends AppCompatActivity implements
    AdapterView.OnItemClickListener {

    private ArrayList<Aluno> alunos;

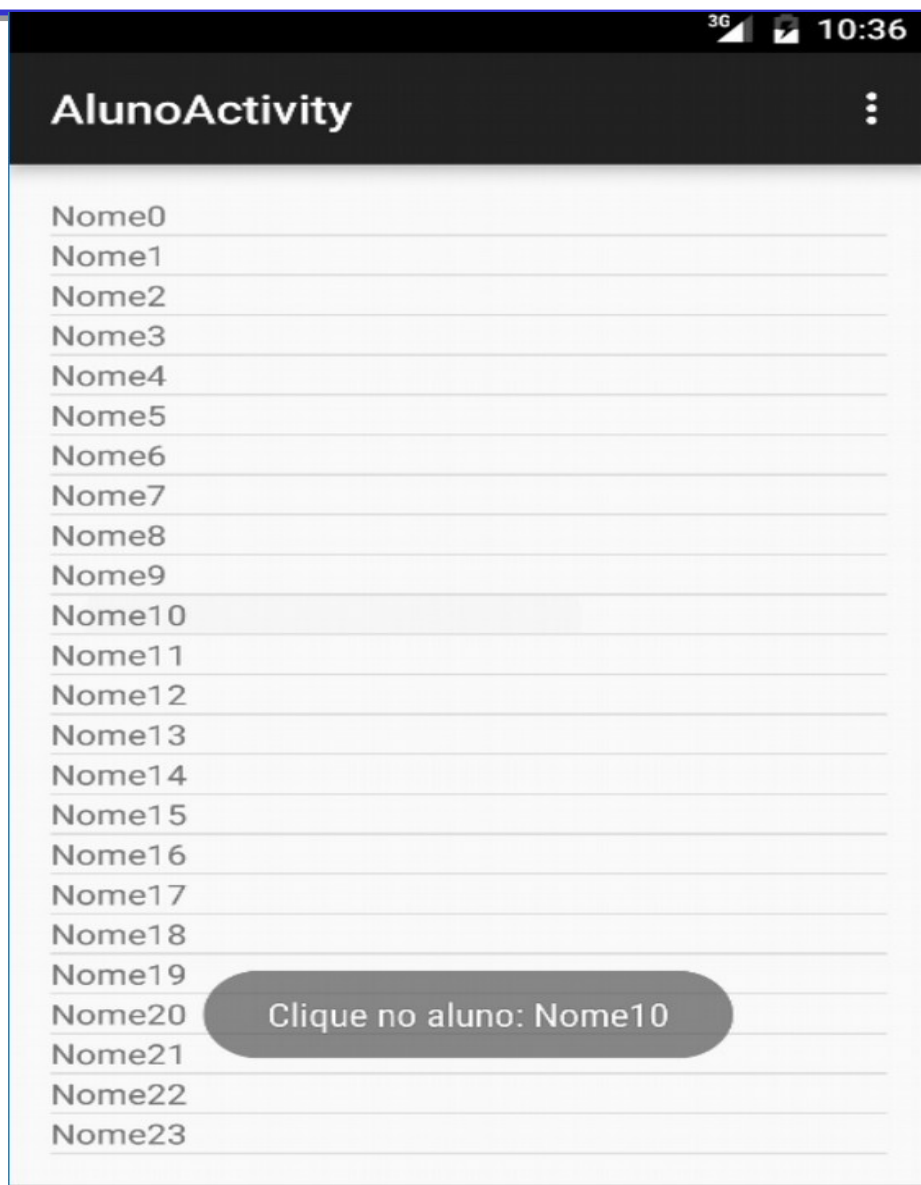
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_aluno);
        ListView listView = (ListView) findViewById(R.id.listView);
        alunos = new ArrayList<Aluno>();
        for(int i=0;i<30;i++){
            Aluno a = new Aluno();
            a.setNome("Nome"+i);
            alunos.add(a);
        }
        listView.setAdapter(new AlunoAdapter(this, alunos));
        listView.setOnItemClickListener(this);
    }

    public void onItemClick(AdapterView<?> parent, View view,
        int position, long id) {

        Aluno a = alunos.get(position);
        Toast.makeText(this, "Clique no aluno: " + a.getNome(),
            Toast.LENGTH_SHORT).show();
    }

}
```

Criando o Listview com Adapter de Alunos



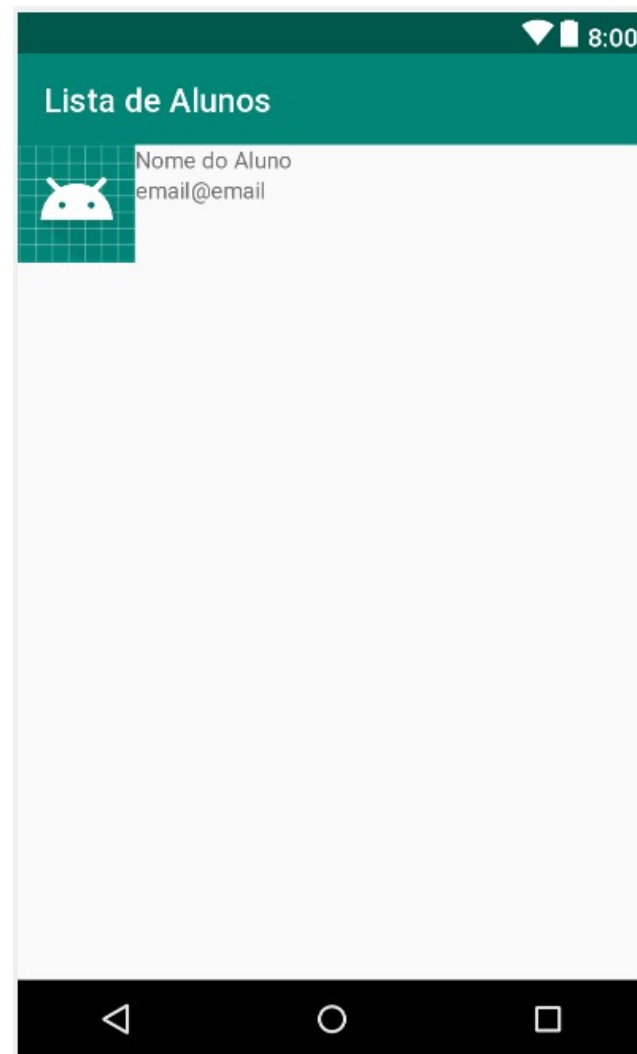
- Um dos métodos mais importantes de BaseAdapter é o `getView()`;
- Pois ele retorna uma View que irá ser apresentada ao usuário na ListView;
- Neste processo, um documento extremamente importante é o documento de layout (arquivo XML) para o item da lista;
- Para usar este layout, vamos precisar de uma Activity, onde teremos acesso ao seu `getLayoutInflater()` – retorna um LayoutInflater;
- Depois de obter o LayoutInflater é possível chamar o método `inflate(id, viewgroup)`, que recebe o id do recurso do arquivo XML desejado e retorna uma instância de um objeto do tipo View;
- Este View permitirá recuperar os demais componentes que fazem parte deste layout.

- Este arquivo de layout representa cada linha a ser exibida na ListView;
- Nosso arquivo de layout contém um LinearLayout, dentro dele temos um ImageView.
- Além disso temos mais um LinearLayout que possui duas TextView, uma para exibir o nome do aluno e outra para exibir o e-mail.

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout
3      xmlns:android="http://schemas.android.com/apk/res/android"
4      xmlns:app="http://schemas.android.com/apk/res-auto"
5      android:layout_width="match_parent"
6      android:layout_height="match_parent"
7      android:orientation="horizontal"
8      android:id="@+id/layout_aluno">
9      <ImageView
10         android:id="@+id/imgAluno"
11         android:layout_width="wrap_content"
12         android:layout_height="wrap_content"
13         app:srcCompat="@mipmap/ic_launcher" />
14      <LinearLayout
15         android:layout_width="match_parent"
16         android:layout_height="match_parent"
17         android:layout_weight="1"
18         android:orientation="vertical">
19          <TextView
20             android:id="@+id/tNomeAluno"
21             android:layout_width="match_parent"
22             android:layout_height="wrap_content"
23             android:text="Nome do Aluno"
24             android:gravity="center_vertical"/>
25          <TextView
26             android:id="@+id/tEmail"
27             android:layout_width="match_parent"
28             android:layout_height="wrap_content"
29             android:text="email@email"
30             android:gravity="bottom"/>
31      </LinearLayout>
32  </LinearLayout>

```



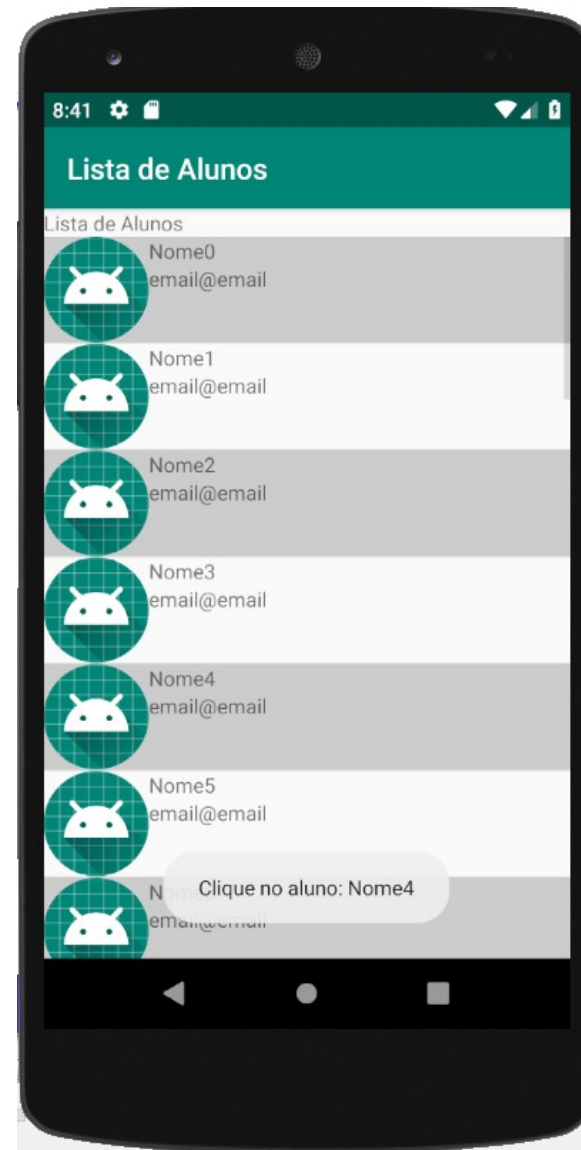
- Altere o método getView() do Adapter para inflar o XML **/res/layout/adapter_aluno.xml**
- Para cada linha do ListView, este XML de layout será utilizado.

```

49 public View getView(int position, View convertView, ViewGroup parent)
50 {
51     LayoutInflater inflater = LayoutInflater.from(context);
52     View view = inflater.inflate(R.layout.adapter_aluno, parent, attachToRoot: false);
53     Aluno a = alunos.get(position);
54     TextView t = (TextView) view.findViewById(R.id.tNomeAluno);
55     ImageView img = (ImageView) view.findViewById(R.id.imgAluno);
56     LinearLayout lay = (LinearLayout) view.findViewById(R.id.layout_aluno);
57     if(position%2==0)
58         lay.setBackgroundColor(Color.parseColor(" #CCCCCC"));
59     t.setText(a.getNome());
60     return view;
61 }
62 }
63

```

Adapter de Alunos



- Utilizada para exibir componentes em formato de grid com linhas e colunas
- Uso mais clássico é exibir várias imagens como em um álbum de fotos
- Na tag `<GridView>` são definidos alguns parâmetros de espaçamento e número de colunas
 - `columnWidth` – Largura de cada coluna do grid. Sempre utilize a notação com dp (density independent pixels)
 - `numColuns` – Número de colunas do grid. No exemplo a seguir, informamos o valor **3**. No entanto, é possível utilizar **auto_fit** para ajustar automaticamente o número de colunas com base na largura da coluna

<RelativeLayout

```
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:paddingLeft="@dimen/activity_horizontal_margin"
android:paddingRight="@dimen/activity_horizontal_margin"
android:paddingTop="@dimen/activity_vertical_margin"
android:paddingBottom="@dimen/activity_vertical_margin"
tools:context="com.example.matth.topaula1.Grid">
```

<GridView

```
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:id="@+id/gridView"
    android:layout_alignParentTop="true"
    android:layout_alignParentLeft="true"
    android:layout_alignParentStart="true"
    android:layout_alignParentBottom="false"
    android:numColumns="3"
    android:layout_alignParentRight="false" />
    <HorizontalScrollView android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:id="@+id/horizontalScrollView"
        android:layout_alignParentTop="true"
        android:layout_alignParentRight="true"
        android:layout_alignParentEnd="true"
        android:layout_marginTop="72dp" />
```

</RelativeLayout>

- XML de layout que será inflado – **adapter_images.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:id="@+id/layImage"
    android:weightSum="1">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textAppearance="? android:attr/textAppearanceSmall"
        android:text="Image Name" android:id="@+id/tImage"
        android:layout_weight="0.07" />
    <ImageView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:id="@+id/imgLabel"
        android:layout_weight="0.24" />
</LinearLayout>
```

- Para exibir as imagens no GridView, é necessário criar um adapter que retorne uma lista com as imagens necessárias.
- Criaremos uma classe de adapter que recebe um array de imagens e cria um ImageView para cada um.
- Para o código compilar, primeiro temos que criar o XML de layout que será inflado pelo código do adapter.

- **ImagesAdapter.java**

```
public class ImagensAdapter extends BaseAdapter {  
    private Context context;  
    private int[] imgs;  
    public ImagensAdapter(Context contex, int[] imgs){  
        super();  
        this.context = contex;  
        this.imgs = imgs;  
    }  
    @Override  
    public int getCount() {  
        return imgs.length;  
    }  
    @Override  
    public Object getItem(int position) {  
        return position;  
    }  
    @Override  
    public long getItemId(int position) {  
        return position;  
    }  
    (...)
```

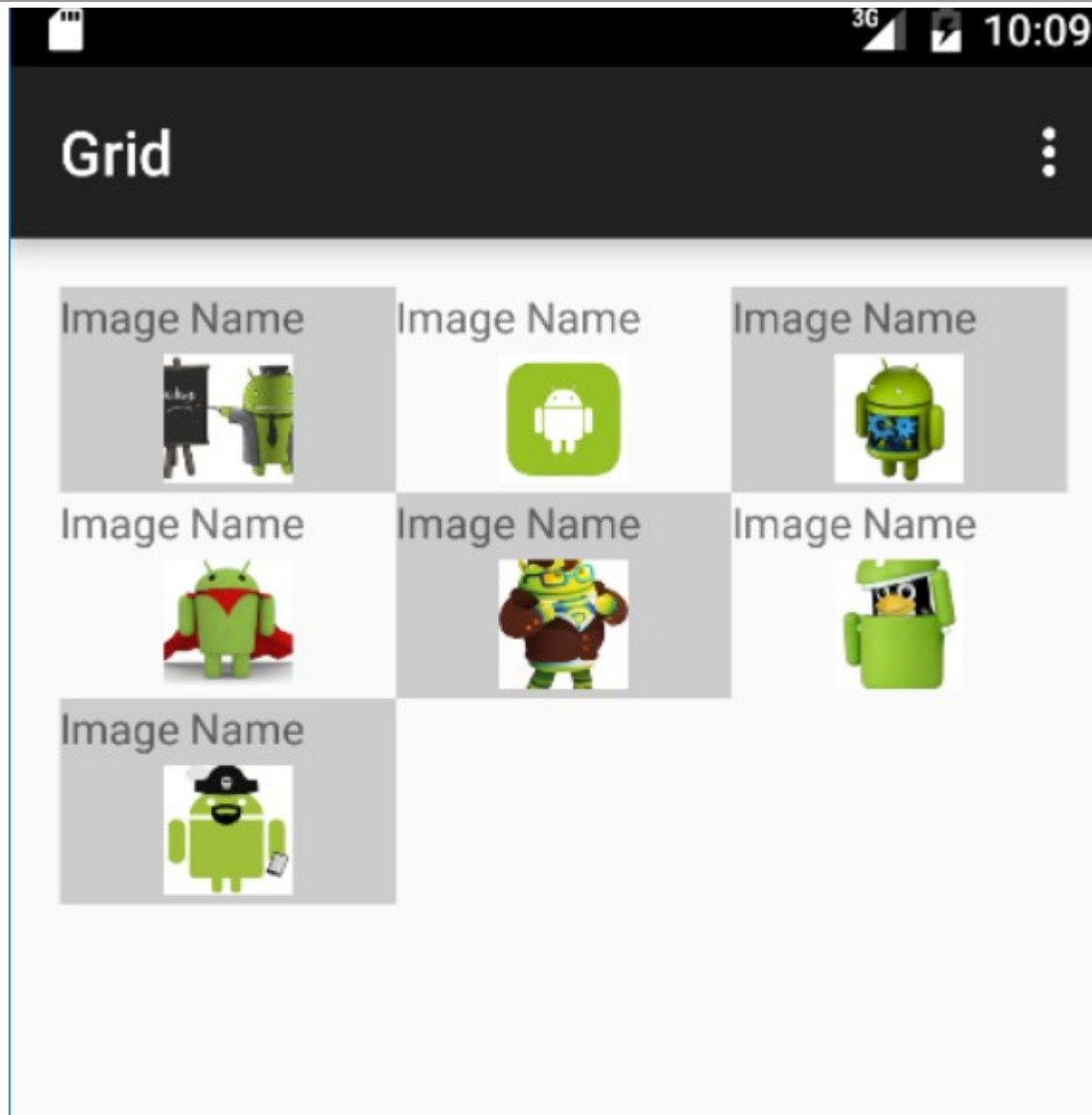
- ImagesAdapter.java (cont)
 - Observe que o método getView deve retornar a View que será inserida em determinada posição do GridView

```
public class ImagensAdapter extends BaseAdapter {
    (...)
    @Override
    public View getView(int position, View convertView, ViewGroup parent) {
        LayoutInflater inflater = LayoutInflater.from(context);
        View view = inflater.inflate(R.layout.adapter_images, parent, false);
        ImageView image = (ImageView) view.findViewById(R.id.imgLabel);
        TextView textimgage = (TextView) view.findViewById(R.id.tImage);
        image.setImageResource(this.imgs[position]);
        LinearLayout lay = (LinearLayout) view.findViewById(R.id.layImage);
        if(position%2==0)
            lay.setBackgroundColor(Color.parseColor("#CCCCCC"));
        return view;
    }
}
```

- Finalmente, segue a activity que vai preencher o GridView com um array de imagens:

```
public class Grid extends AppCompatActivity implements
    AdapterView.OnItemClickListener {

    private GridView gridView;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_grid);
        gridView = (GridView) findViewById(R.id.gridView);
        int[] images = new int[]{
            R.mipmap.img, R.mipmap.img1, R.mipmap.img2, R.mipmap.img3,
            R.mipmap.img4, R.mipmap.img5, R.mipmap.img6
        };
        gridView.setAdapter(new ImagensAdapter(this, images));
        gridView.setOnItemClickListener(this);
    }
    @Override
    public void onItemClick(AdapterView<?> parent, View view, int position,
        long id) {
        Toast.makeText(this, "Imagem selecionada: " + position,
            Toast.LENGTH_SHORT).show();
    }
}
```



- Álbum de fotos no Android
- Gesto de “swipe” (deslizar) para os lados para ver as fotos
- Componente: `android.widget.Gallery`
- A forma de usar o Gallery é idêntica ao GridView
- Pode-se usar o mesmo adapter (**ImagesAdapter**) criado para o GridView para inserir dados na Gallery

```
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    android:paddingBottom="@dimen/activity_vertical_margin"
    tools:context="com.example.matth.topaula1.galeria">
    <Gallery android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:id="@+id/gallery"
        android:layout_alignParentTop="true"
        android:layout_alignParentLeft="true"
        android:layout_alignParentStart="true" />
</RelativeLayout>
```

```
public class galeria extends AppCompatActivity implements
    AdapterView.OnItemClickListener {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_galeria);
        int[] images = new int[]{
            R.mipmap.img, R.mipmap.img1, R.mipmap.img2, R.mipmap.img3,
            R.mipmap.img4, R.mipmap.img5, R.mipmap.img6
        };
        Gallery g = (Gallery) findViewById(R.id.gallery);
        g.setAdapter(new ImagensAdapter(this, images));
        g.setOnItemClickListener(this);
    }

    @Override
    public void onItemClick(AdapterView<?> parent, View view, int position,
        long id) {
        Toast.makeText(this, "Imagem selecionada: " + position,
            Toast.LENGTH_SHORT).show();
    }
}
```

- Recentemente a classe Gallery foi descontinuada (deprecated)
- A utilização da classe **ViewPager** possui o mesmo comportamento e é bem mais flexível.
- A classe `android.support.v4.view.ViewPager` faz parte da biblioteca de compatibilidade
- A biblioteca de compatibilidade é baixada pelo SDK Manager pelo item Android Support Library e recomenda-se sempre mantê-la atualizada.

```
<RelativeLayout
```

```
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    android:paddingBottom="@dimen/activity_vertical_margin"
    tools:context="com.example.matth.topaula1.TesteViewPager">
```

```
    <android.support.v4.view.ViewPager
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:id="@+id/viewpager"
        android:layout_alignParentTop="true"
        android:layout_alignParentLeft="true"
        android:layout_alignParentStart="true" />
```

```
</RelativeLayout>
```

- Para preencher as views do ViewPager, deve-se informar um tipo especial de adapter, que é uma implementação da classe `android.support.v4.ViewPager`
- Lembre-se que um adapter apenas retorna uma view para determinada posição
- Usaremos o mesmo layout já criado para o GridView e o Gallery para inflar os elementos na view, no entanto será preciso escrever um adapter para o ViewPager

Método	Descrição
getCount	Retorna a quantidade de elementos do adapter
instantiateItem	Retorna um objeto chave chamado key-object, que é usado para controlar o ViewPager
isViewFromObject	Validar se a view informada como parâmetro corresponde ao objeto-chave informado
destroyItem	Utilizado para destruir a view associado a um objeto-chave.

```
public class ViewPagerAdapter extends PagerAdapter {
    private Context context;
    private int[] images;
    public ViewPagerAdapter(Context context, int[] images){
        this.context = context;
        this.images = images;
    }
    @Override
    public int getCount() {
        return images != null ? images.length : 0;
    }
    @Override
    public boolean isViewFromObject(View view, Object object) {
        return view == object;
    }
    @Override
    public Object instantiateItem(ViewGroup container, int position) {
        View view = LayoutInflater.from(context).inflate(R.layout.adapter_images,
                                                         container, false);

        ImageView img = (ImageView) view.findViewById(R.id.imgLabel);
        img.setImageResource(images[position]);
        ((ViewGroup) container).addView(view);
        return view;
    }
    @Override
    public void destroyItem(ViewGroup container, int position, Object view) {
        ((ViewPager)container).removeView((View) view);
    }
}
```